

Sec 5.1. Networking

A. Networking Concepts

Task 1 - OSI Model

The OSI model is composed of seven layers:

1. Physical Layer
2. Data Link Layer
3. Network Layer
4. Transport Layer
5. Session Layer
6. Presentation Layer
7. Application Layer

Layer 1: Physical Layer: The physical layer, also referred to as layer 1, deals with the physical connection between devices; this includes the medium, such as a wire, and the definition of the binary digits 0 and 1. Data transmission can be via an electrical, optical, or wireless signal. Consequently, we need data cables or antennas, depending on our physical medium.

Layer 2: Data Link Layer: The physical layer defines a medium to transmit our signal. The data link layer, i.e., layer 2, represents the protocol that enables data transfer between nodes on the same network segment. The data link layer describes an agreement between the different systems on the same network segment on how to communicate. A network segment refers to a group of networked devices using a shared medium or channel for information transfer. For example, consider a company office with ten computers connected to a network switch; that's a network segment. Examples of layer 2 include Ethernet, i.e., 802.3, and WiFi, i.e., 802.11. Ethernet and WiFi addresses are six bytes. Their address is called a MAC address, where MAC stands for Media Access Control. They are usually expressed in hexadecimal format with a colon separating each two hexadecimal digits (one byte). The three leftmost bytes identify the vendor.

Layer 3: Network Layer: The data link layer focuses on sending data between two nodes on the same network segment. The network layer, i.e., layer 3, is concerned with sending data between different networks. In more technical terms, the network layer handles logical addressing and routing, i.e., finding a path to transfer the network packets between the diverse networks. In the data link layer, we gave an example of one company office with ten computers, where the data link layer is responsible for providing a connection between them. Let's say that this company has multiple offices distributed across various cities, countries, or even continents. The network layer is responsible for connecting the different offices together.

Layer 4: Transport Layer: Layer 4, the transport layer, enables end-to-end communication between running applications on different hosts. Your web browser is connected to the TryHackMe web server over the transport layer, which can support various functions like flow control, segmentation, and error correction.

Layer 5: Session Layer: The session layer is responsible for establishing, maintaining, and synchronising communication between applications running on different hosts. Establishing a session means initiating communication between applications and negotiating the necessary parameters for the session. Data synchronisation ensures that data is transmitted in the correct order and provides mechanisms for recovery in case of transmission failures.

Layer 6: Presentation Layer: The presentation layer ensures the data is delivered in a form the application layer can understand. Layer 6 handles data encoding, compression, and encryption. An example of encoding is character encoding, such as ASCII or Unicode. Various standards are used at the presentation layer. Consider the scenario where we want to send an image via email.

Layer 7: Application Layer: The application layer provides network services directly to end-user applications. Your web browser would use the HTTP protocol to request a file, submit a form, or upload a file. The application layer is the top layer, and you might have encountered many of its protocols as you use different applications. Examples of Layer 7 protocols are HTTP, FTP, DNS, POP3, SMTP, and IMAP.

Layer Number	Layer Name	Main Function	Example Protocols and Standards
Layer 7	Application layer	Providing services and interfaces to applications	HTTP, FTP, DNS, POP3, SMTP, IMAP

Layer Number	Layer Name	Main Function	Example Protocols and Standards
Layer 6	Presentation layer	Data encoding, encryption, and compression	Unicode, MIME, JPEG, PNG, MPEG
Layer 5	Session layer	Establishing, maintaining, and synchronising sessions	NFS, RPC
Layer 4	Transport layer	End-to-end communication and data segmentation	UDP, TCP
Layer 3	Network layer	Logical addressing and routing between networks	IP, ICMP, IPSec
Layer 2	Data link layer	Reliable data transfer between adjacent nodes	Ethernet (802.3), WiFi (802.11)
Layer 1	Physical layer	Physical data transmission media	Electrical, optical, and wireless signals

Answer the questions below

- Which layer is responsible for end-to-end communication between running applications? 4
- Which layer is responsible for routing packets to the proper network? 3
- In the OSI model, which layer is responsible for encoding the application data? 6
- Which layer is responsible for transferring data between hosts on the same network segment? 2

Task 2 - TCP/IP Model

- **Application Layer:** The OSI model application, presentation and session layers, i.e., layers 5, 6, and 7, are grouped into the application layer in the TCP/IP model.

- **Transport Layer:** This is layer 4.
- **Internet Layer:** This is layer 3. The OSI model's network layer is called the Internet layer in the TCP/IP model.
- **Link Layer:** This is layer 2.

Layer Number	ISO OSI Model	TCP/IP Model (RFC 1122)	Protocols
7	Application Layer	Application Layer	HTTP, HTTPS, FTP, POP3, SMTP, IMAP, Telnet, SSH,
6	Presentation Layer		
5	Session Layer		
4	Transport Layer	Transport Layer	TCP, UDP
3	Network Layer	Internet Layer	IP, ICMP, IPSec
2	Data Link Layer	Link Layer	Ethernet 802.3, WiFi 802.11
1	Physical Layer		

Answer the questions below

To which layer does HTTP belong in the TCP/IP model? Application Model

How many layers of the OSI model does the application layer in the TCP/IP model cover? 3

Task 3 - IP Addresses and Subnets

An IP (Internet Protocol) address is a unique numerical identifier assigned to a device on a network. It allows devices to communicate with each other, similar to how a postal address helps mail reach the correct destination.

A subnet (subnetwork) is a logical division of an IP network. Subnetting helps: Improve network performance, Reduce broadcast traffic, Increase security, Organize devices into separate network segments. A subnet determines: Which part of an IP address identifies the network, Which part identifies the host (device)

Subnet Mask: A subnet mask separates the network portion from the host portion of an IP address.

- **IP Address** = Unique identifier for a device on a network.
- **Subnet** = Smaller network created from a larger network.
- **Subnet Mask/CIDR** = Defines network and host portions of an IP address.
- **Subnetting** = Improves security, organization, and network efficiency.

Looking Up Your Network Configuration: Windows command line using the command `ipconfig`. On Linux and UNIX-based systems `ifconfig` or `ip address show`, which can be typed as `ip a s`. If you are wondering, a subnet mask of `255.255.255.0` can also be written as `/24`. The `/24` means that the leftmost 24 bits within the IP address do not change across the network, i.e., the subnet. In other words, the leftmost three octets are the same across the whole subnet; therefore, we can expect to find addresses that range from `192.168.66.1` to `192.168.66.254`. `192.168.66.0` and `192.168.66.255` are the network and broadcast addresses, respectively.

"/24" is CIDR notation that represents a subnet mask of 255.255.255.0. It means the first 24 bits of the IP address are used for the network portion, and the remaining 8 bits are used for hosts. Since 8 bits are available for hosts, the subnet contains 256 IP addresses (2^8), of which 254 are usable because one address is reserved for the network and one for the broadcast address. CIDR (Classless Inter-Domain Routing) is a method of representing IP addresses and subnet masks more efficiently than the older class-based system (Class A, B, C).

As we are explaining IP addresses, it is useful to mention that for most practical purposes, there are two types of IP addresses:

- Public IP addresses
- Private IP addresses

RFC 1918 defines the following three ranges of private IP addresses: (RFC stands for Request for Comments. It is a formal document that describes Internet standards, protocols, and technical specifications. Many networking technologies, such as TCP/IP, HTTP, DNS, and SMTP, are defined in RFC documents.)

- `10.0.0.0` - `10.255.255.255` (`10/8`)
- `172.16.0.0` - `172.31.255.255` (`172.16/12`)
- `192.168.0.0` - `192.168.255.255` (`192.168/16`)

Routing: A router forwards data packets to the proper network. Usually, a data packet passes through multiple routers before it reaches its final destination. The router functions at layer 3, inspecting the IP address and forwarding the packet to the best network (router) so the packet gets closer to its destination.

Answer the questions below

| Which of the following IP addresses is not a private IP address?

192.168.250.125

10.20.141.132

49.69.147.197 *

172.23.182.251

| Which of the following IP addresses is not a valid IP address?

192.168.250.15

192.168.254.17

192.168.305.19 *

192.168.199.13

Task 4 - UDP and TCP

UDP (User Datagram Protocol) allows us to reach a specific process on this target host. UDP is a simple connectionless protocol that operates at the transport layer, i.e., layer 4. Being connectionless means that it does not need to establish a connection. UDP does not even provide a mechanism to know that the packet has been delivered. A real-life example similar to UDP is the standard mail service, with no delivery confirmation. In other words, there is no guarantee that the UDP packet has been received successfully.

TCP (Transmission Control Protocol) is a connection-oriented transport protocol. It uses various mechanisms to ensure reliable data delivery sent by the different processes on the networked hosts. Like UDP, it is a layer 4 protocol. Being connection-oriented, it requires the establishment of a TCP connection before any data can be sent. In TCP, each data octet has a sequence number; this makes it easy for the receiver to identify lost or duplicated packets. The receiver, on the other hand, acknowledges the reception of data with an acknowledgement number specifying the last received octet. A TCP connection is established using what's

called a three-way handshake. Two flags are used: SYN (Synchronise) and ACK (Acknowledgment). The packets are sent as follows:

1. SYN Packet: The client initiates the connection by sending a SYN packet to the server. This packet contains the client's randomly chosen initial sequence number.
2. SYN-ACK Packet: The server responds to the SYN packet with a SYN-ACK packet, which adds the initial sequence number randomly chosen by the server.
3. ACK Packet: The three-way handshake is completed as the client sends an ACK packet to acknowledge the reception of the SYN-ACK packet.

Similar to UDP, TCP identifies the process of initiating or waiting (listening) for a connection using port numbers. As stated, a valid port number ranges between 1 and 65535 because it uses two octets and port 0 is reserved.

Answer the questions below

| Which protocol requires a three-way handshake? TCP

| What is the approximate number of port numbers (in thousands)? 65

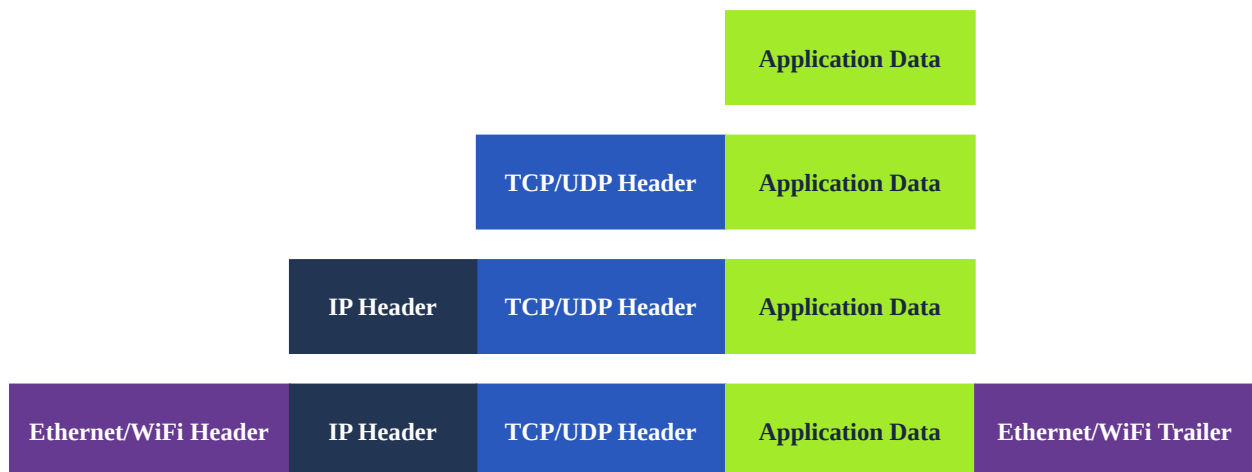
Task 5 - Encapsulation

Encapsulation refers to the process of every layer adding a header (and sometimes a trailer) to the received unit of data and sending the "encapsulated" unit to the layer below. Encapsulation is an essential concept as it allows each layer to focus on its intended function. In the image below, we have the following four steps:

- **Application data:** It all starts when the user inputs the data they want to send into the application. For example, you write an email or an instant message and hit the send button. The application formats this data and starts sending it according to the application protocol used, using the layer below it, the transport layer.
- **Transport protocol segment or datagram:** The transport layer, such as TCP or UDP, adds the proper header information and creates the TCP **segment** (or UDP **datagram**). This segment is sent to the layer below it, the network layer.

- **Network packet:** The network layer, i.e. the Internet layer, adds an IP header to the received TCP segment or UDP datagram. Then, this IP **packet** is sent to the layer below it, the data link layer.
- **Data link frame:** The Ethernet or WiFi receives the IP packet and adds the proper header and trailer, creating a **frame**.

We start with application data. At the transport layer, we add a TCP or UDP header to create a **TCP segment** or **UDP datagram**. Again, at the network layer, we add the proper IP header to get an **IP packet** that can be routed over the Internet. Finally, we add the appropriate header and trailer to get a WiFi or Ethernet frame at the link layer.



The process has to be reversed on the receiving end until the application data is extracted.

Answer the questions below

- | On a WiFi, within what will an IP packet be encapsulated? Frame
- | What do you call the UDP data unit that encapsulates the application data?
Datagram
- | What do you call the data unit that encapsulates the application data sent over TCP? Segment

Task 6 - Telnet

The TELNET (Teletype Network) protocol is a network protocol for remote terminal connection. In simpler words, `telnet`, a TELNET client, allows you to connect to and communicate with a remote system and issue text commands. Although initially it was used for remote administration, we can use `telnet` to connect to any server listening on a TCP port number. On the target lab machine, different services are running. We will experiment with three of them:

- Echo server: This server echoes everything you send it. By default, it listens on port 7.
- Daytime server: This server listens on port 13 by default and replies with the current day and time.
- Web (HTTP) server: This server listens on TCP port 80 by default and serves web pages.

Before continuing, we should mention that the echo and daytime servers are considered security risks and should not be run; however, we started them explicitly to demonstrate communication with the server using `telnet`. In the terminal below, we connect to the target VM at the echo server's TCP port number 7. To close the connection, press the `CTRL` + `I` keys simultaneously.

```
user@TryHackMe$ telnet MACHINE_IP 7
telnet MACHINE_IP 7
Trying MACHINE_IP...
Connected to MACHINE_IP.
Escape character is '^]'.
Hi
Hi
How are you?
How are you?
Bye
Bye
^]

telnet> quit
Connection closed.
```

In the terminal below, we use `telnet` to connect to the daytime server listening at port 13. We noticed that the connection closes once the current date and time are returned.

```
user@TryHackMe$ telnet MACHINE_IP 13
Trying MACHINE_IP...
Connected to MACHINE_IP.
Escape character is '^]'.
Thu Jun 20 12:36:32 PM UTC 2024
Connection closed by foreign host.
```

Finally, let's request a web page using `telnet`. After connecting to port 80, you need to issue the command `GET / HTTP/1.1` and identify the host where anything goes, such as `Host: telnet.thm`. Next, you need to press `Enter` twice so your last input line is a blank line. The output below shows the exchange. (The page has been redacted.)

Note: You may have to press `Enter` after sending the information in case you don't get a response.

```
user@TryHackMe$ telnet MACHINE_IP 80
Trying MACHINE_IP...
Connected to MACHINE_IP.
Escape character is '^]'.
GET / HTTP/1.1
Host: telnet.thm

HTTP/1.1 200 OK
Content-Type: text/html
[...]

Connection closed by foreign host.
```

B. Networking Essentials

Task 1 - DHCP

DHCP is a protocol that automatically assigns IP addresses and network configuration parameters to devices on a network. It simplifies network administration by eliminating the need to manually configure IP settings on each device. DHCP follows four steps: Discover, Offer, Request, and Acknowledge (DORA):

1. **DHCP Discover:** The client broadcasts a DHCPDISCOVER message seeking the local DHCP server if one exists.
2. **DHCP Offer:** The server responds with a DHCPOFFER message with an IP address available for the client to accept.
3. **DHCP Request:** The client responds with a DHCPREQUEST message to indicate that it has accepted the offered IP.
4. **DHCP Acknowledge:** The server responds with a DHCPACK message to confirm that the offered IP address is now assigned to this client.

The client starts without any IP network configuration. It only has a MAC address. In the first and third packets, DHCP Discover and DHCP Request, the client searching for a DHCP server still has no IP network configuration and has not yet used the DHCP server's offered IP address. Therefore, it sends packets from the IP address 0.0.0.0 to the broadcast IP address 255.255.255.255.

Answer the questions below

How many steps does DHCP use to provide network configuration? 4

What is the destination IP address that a client uses when it sends a DHCP Discover packet? 255.255.255.255

What is the source IP address a client uses when trying to get IP network configuration over DHCP? 0.0.0.0

Task 2 - ARP

Address Resolution Protocol (ARP) makes it possible to find the MAC address of another device on the Ethernet. In the example below, a host with the IP address 192.168.66.89 wants to communicate with another system with the IP address 192.168.66.1. It sends an ARP Request asking the host with the IP address 192.168.66.1 to respond. The ARP Request is sent from the MAC address of

the requester to the broadcast MAC address, `ff:ff:ff:ff:ff:ff` as shown in the first packet. The ARP Reply arrived shortly afterwards, and the host with the IP address `192.168.66.1` responded with its MAC address. From this point, the two hosts can exchange data link layer frames.

Task 3 - ICMP

Internet Control Message Protocol (ICMP) is mainly used for network diagnostics and error reporting. Two popular commands rely on ICMP, and they are instrumental in network troubleshooting and network security. The commands are:

- `ping`: This command uses ICMP to test connectivity to a target system and measures the round-trip time (RTT). In other words, it can be used to learn that the target is alive and that its reply can reach our system.
- `tracert`: This command is called `tracert` on Linux and UNIX-like systems and `tracert` on MS Windows systems. It uses ICMP to discover the route from your host to the target.

Ping: Ping is a network utility that tests connectivity between two devices by sending ICMP Echo Request messages and waiting for ICMP Echo Reply responses. It is used to verify if a host is reachable and to measure round-trip time. In the example below, we used `-c 4` to tell the `ping` command to stop after sending four packets.

```
user@TryHackMe$ ping 192.168.11.1 -c 4
PING 192.168.11.1 (192.168.11.1) 56(84) bytes of data.
64 bytes from 192.168.11.1: icmp_seq=1 ttl=63 time=11.2 ms
64 bytes from 192.168.11.1: icmp_seq=2 ttl=63 time=3.81 ms
64 bytes from 192.168.11.1: icmp_seq=3 ttl=63 time=3.99 ms
64 bytes from 192.168.11.1: icmp_seq=4 ttl=63 time=23.4 ms

--- 192.168.11.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3003ms
rtt min/avg/max/mdev = 3.805/10.596/23.366/7.956 ms
```

The output shows no packet loss; moreover, it calculates the minimum, average, maximum, and standard deviation (mdev) of the round-trip time (RTT).

Traceroute: Traceroute is a network troubleshooting tool that shows the route packets take from a source to a destination. It identifies each router (hop) along the path and measures the response time at each hop, helping administrators diagnose connectivity and routing problems.

```
user@TryHackMe$ traceroute example.com
traceroute to example.com (93.184.215.14), 30 hops max, 60 byte packets
 1  _gateway (192.168.66.1)  4.414 ms  4.342 ms  4.320 ms
 2  192.168.11.1 (192.168.11.1)  5.849 ms  5.830 ms  5.811 ms
 3  100.104.0.1 (100.104.0.1)  11.130 ms  11.111 ms  11.093 ms
 4  10.149.1.45 (10.149.1.45)  6.156 ms  6.138 ms  6.120 ms
 5  * * *
 6  * * *
 7  * * *
 8  172.16.48.1 (172.16.48.1)  5.667 ms  8.165 ms  6.861 ms
 9  ae81.edge4.Marseille1.Level3.net (212.73.201.45)  50.811 ms
10  52.857 ms 213.242.116.233 (213.242.116.233)  52.798 ms
11  NTT-level3-Marseille1.Level3.net (4.68.68.150)  93.351 ms
12  79.897 ms 79.804 ms
13  ae-9.r20.parsfr04.fr.bb.gin.ntt.net (129.250.3.38)  62.935 ms
14  62.908 ms 64.313 ms
15  ae-14.r21.nwrknj03.us.bb.gin.ntt.net (129.250.4.194)  141.816 ms
16  141.782 ms 141.757 ms
17  ae-1.a02.nycmny17.us.bb.gin.ntt.net (129.250.3.17)  145.786 ms
18  6 ms ae-1.a03.nycmny17.us.bb.gin.ntt.net (129.250.3.128)  141.701 ms
19  147.586 ms
20  ce-0-3-0.a02.nycmny17.us.ce.gin.ntt.net (128.241.1.14)  148.692 ms
21  ce-3-3-0.a03.nycmny17.us.ce.gin.ntt.net (128.241.1.90)  141.615 ms
22  ce-0-3-0.a02.nycmny17.us.ce.gin.ntt.net (128.241.1.14)  148.168 ms
23  ae-66.core1.nyd.edgecastcdn.net (152.195.69.133)  141.100 ms
24  ae-65.core1.nyd.edgecastcdn.net (152.195.68.133)  140.360 ms
25  ae-66.core1.nyd.edgecastcdn.net (152.195.69.133)  140.638 ms
26  93.184.215.14 (93.184.215.14)  140.574 ms 140.543 ms 140.514 ms
```

```
17  93.184.215.14 (93.184.215.14)  140.488 ms  139.397 ms  14
1.854 ms
```

Answer the questions below

Using the example images above, how many bytes were sent in the echo (ping) request? 40

Which IP header field does the traceroute command require to become zero? TTL

Task 4 - Routing

Routing is the process of forwarding network traffic between different networks. Routers examine the destination IP address of a packet and use routing tables to determine the best path to deliver it to its destination.

- **OSPF (Open Shortest Path First):** OSPF is a routing protocol that allows routers to share information about the network topology and calculate the most efficient paths for data transmission. It does this by having routers exchange updates about the state of their connected links and networks. This way, each router has a complete map of the network and can determine the best routes to reach any destination.
- **EIGRP (Enhanced Interior Gateway Routing Protocol):** EIGRP is a Cisco proprietary routing protocol that combines aspects of different routing algorithms. It allows routers to share information about the networks they can reach and the cost (like bandwidth or delay) associated with those routes. Routers then use this information to choose the most efficient paths for data transmission.
- **BGP (Border Gateway Protocol):** BGP is the primary routing protocol used on the Internet. It allows different networks (like those of Internet Service Providers) to exchange routing information and establish paths for data to travel between these networks. BGP helps ensure data can be routed efficiently across the Internet, even when traversing multiple networks.
- **RIP (Routing Information Protocol):** RIP is a simple routing protocol often used in small networks. Routers running RIP share information about the networks they can reach and the number of hops (routers) required to get there. As a

result, each router builds a routing table based on this information, choosing the routes with the fewest hops to reach each destination.

Which routing protocol discussed in this task is a Cisco proprietary protocol?
EIGRP

Task 5 - NAT

NAT is a process used in networking where private IP addresses are translated into a public IP address so devices inside a private network can access the internet. It helps conserve public IP addresses and adds a layer of security by hiding internal IPs.

C. Networking Core Protocols

Task 1 - DNS

DNS (Domain Name System) is a system that translates domain names like google.com into IP addresses so computers can locate each other on the network. It works through a hierarchy of DNS servers and uses records like A, CNAME, and MX to map domains to different services. DNS is essential for how the internet functions. DNS operates at the Application Layer, i.e., Layer 7 of the ISO OSI model. DNS traffic uses UDP port 53 by default and TCP port 53 as a default fallback. There are many types of DNS records; however, in this task, we will focus on the following four:

- **A record:** The A (Address) record maps a hostname to one or more IPv4 addresses. For example, you can set `example.com` to resolve to `172.17.2.172`.
- **AAAA Record:** The AAAA record is similar to the A Record, but it is for IPv6. Remember that it is AAAA (quad-A), as AA and AAA would refer to a battery size; furthermore, AAA refers to *Authentication, Authorization, and Accounting*; neither falls under DNS.
- **CNAME Record:** The CNAME (Canonical Name) record maps a domain name to another domain name. For example, `www.example.com` can be mapped to `example.com` or even to `example.org`.

- **MX Record:** The MX (Mail Exchange) record specifies the mail server responsible for handling emails for a domain.

`nslookup` is a network utility used to query DNS servers and retrieve domain name or IP address information. It is commonly used for troubleshooting DNS resolution problems and verifying DNS records.

```
user@TryHackMe$ nslookup www.example.com
Server:          127.0.0.53
Address:         127.0.0.53#53

Non-authoritative answer:
Name:   www.example.com
Address: 93.184.215.14
Name:   www.example.com
Address: 2606:2800:21f:cb07:6820:80da:af6b:8b2c
```

| Which DNS record type refers to IPv6? AAAA

| Which DNS record type refers to the email server? MX

Task 2 - WHOIS

WHOIS is a protocol used to look up registration details of a domain name or IP address, including the owner, registrar, creation and expiry dates, and name servers. It is commonly used in cybersecurity for OSINT and investigation purposes.

Task 3 - HTTP(S)

HTTP stands for Hypertext Transfer Protocol; the S in HTTPS stands for Secure. This protocol relies on TCP and defines how your web browser communicates with the web servers. Some of the commands or methods that your web browser commonly issues to the web server are:

- `GET` retrieves data from a server, such as an HTML file or an image.
- `POST` allows us to submit new data to the server, such as submitting a form or uploading a file.

- **PUT** is used to create a new resource on the server and to update and overwrite existing information.
- **DELETE**, as the name suggests, is used to delete a specified file or resource on the server.

HTTP and HTTPS commonly use TCP ports 80 and 443, respectively, and less commonly other ports such as 8080 and 8443.

Task 4 - FTP

File Transfer Protocol (FTP) is designed to transfer files. As a result, FTP is very efficient for file transfer, and when all conditions are equal, it can achieve higher speeds than HTTP. Example commands defined by the FTP protocol are:

- **USER** is used to input the username
- **PASS** is used to enter the password
- **RETR** (retrieve) is used to download a file from the FTP server to the client.
- **STOR** (store) is used to upload a file from the client to the FTP server.

FTP server listens on TCP port 21 by default; data transfer is conducted via another connection from the client to the server.

In the terminal below we executed the command `ftp MACHINE_IP` to connect to the remote FTP server using the local `ftp` client. Then we went through the following steps:

```
user@TryHackMe$ ftp MACHINE_IP
Connected to MACHINE_IP (MACHINE_IP).
220 (vsFTPd 3.0.5)
Name (MACHINE_IP:strategos): anonymous
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> ls
227 Entering Passive Mode (10,10,41,192,134,10).
```

```
150 Here comes the directory listing.
-rw-r--r--    1 0      0      1480 Jun 27 08:03 coffe
e.txt
-rw-r--r--    1 0      0      14 Jun 27 08:04 flag.t
xt
-rw-r--r--    1 0      0     1595 Jun 27 08:05 tea.tx
t
226 Directory send OK.
ftp> type ascii
200 Switching to ASCII mode.
ftp> get coffee.txt
local: coffee.txt remote: coffee.txt
227 Entering Passive Mode (10,10,41,192,57,100).
150 Opening BINARY mode data connection for coffee.txt (1480 b
ytes).
WARNING! 47 bare linefeeds received in ASCII mode
File may not have transferred correctly.
226 Transfer complete.
1480 bytes received in 8e-05 secs (18500.00 Kbytes/sec)
ftp> quit
221 Goodbye.
```

Task 5 - SMTP

As with browsing the web and downloading files, sending email needs its own protocol. Simple Mail Transfer Protocol (SMTP) defines how a mail client talks with a mail server and how a mail server talks with another. The analogy for the SMTP protocol is when you go to the local post office to send a package. You greet the employee, tell them where you want to send your package, and provide the sender's information before handing them the package. Depending on the country you are in, you might be asked to show your identity card. This process is not very different from an SMTP session. The terminal below shows an example of an email sent via `telnet`. The SMTP server listens on TCP port 25 by default.

```
user@TryHackMe$ telnet MACHINE_IP 25
Trying MACHINE_IP...
```

```
Connected to MACHINE_IP.
Escape character is '^]'.
220 example.thm ESMTP Exim 4.95 Ubuntu Thu, 27 Jun 2024 16:18:
09 +0000
HELO client.thm
250 example.thm Hello client.thm [10.11.81.126]
MAIL FROM: <user@client.thm>
250 OK
RCPT TO: <strategos@server.thm>
250 Accepted
DATA
354 Enter message, ending with "." on a line by itself
From: user@client.thm
To: strategos@server.thm
Subject: Telnet email

Hello. I am using telnet to send you an email!
.
250 OK id=1sMrpq-0001Ah-UT
QUIT
221 example.thm closing connection
Connection closed by foreign host.
```

Which SMTP command indicates that the client will start the contents of the email message? DATA

What does the email client send to indicate that the email message has been fully entered? .

Let's present some of the commands used by your mail client when it transfers an email to an SMTP server:

- **HELO** or **EHLO** initiates an SMTP session
- **MAIL FROM** specifies the sender's email address
- **RCPT TO** specifies the recipient's email address

- **DATA** indicates that the client will begin sending the content of the email message
- **.** is sent on a line by itself to indicate the end of the email message

Task 6 - POP3

You've received an email and want to download it to your local mail client. The Post Office Protocol version 3 (POP3) is designed to allow the client to communicate with a mail server and retrieve email messages. an email client sends its messages by relying on SMTP and retrieves them using POP3. SMTP is similar to handing your envelope or package to the post office, and POP3 is similar to checking your local mailbox for new letters or packages. the POP3 server listens on TCP port 110 by default, the command to connect to the TELNET port is `telnet MACHINE_IP 110`.

```
user@TryHackMe$ telnet MACHINE_IP 110
Trying MACHINE_IP...
Connected to MACHINE_IP.
Escape character is '^]'.
+OK [XCLIENT] Dovecot (Ubuntu) ready.
AUTH
+OK
PLAIN
.
USER strategos
+OK
PASS
+OK Logged in.
STAT
+OK 3 1264
LIST
+OK 3 messages:
1 407
2 412
3 445
.
RETR 3
```

```
+OK 445 octets
Return-path: <user@client.thm>
Envelope-to: strategos@server.thm
Delivery-date: Thu, 27 Jun 2024 16:19:35 +0000
Received: from [10.11.81.126] (helo=client.thm)
    by example.thm with smtp (Exim 4.95)
    (envelope-from <user@client.thm>)
    id 1sMrpq-0001Ah-UT
    for strategos@server.thm;
    Thu, 27 Jun 2024 16:19:35 +0000
From: user@client.thm
To: strategos@server.thm
Subject: Telnet email

Hello. I am using telnet to send you an email!
.
QUIT
+OK Logging out.
Connection closed by foreign host.
```

Task 7 - IMAP

POP3 is enough when working from one device, e.g., your favourite email client on your desktop computer. However, what if you want to check your email from your office desktop computer and from your laptop or smartphone? In this scenario, you need a protocol that allows synchronization of messages instead of deleting a message after retrieving it. One solution to maintaining a synchronized mailbox across multiple devices is Internet Message Access Protocol (IMAP).

IMAP allows synchronizing read, moved, and deleted messages. IMAP is quite convenient when you check your email via multiple clients. Unlike POP3, which tends to minimize server storage as email is downloaded and deleted from the remote server, IMAP tends to use more storage as email is kept on the server and synchronized across the email clients. The IMAP protocol commands are more complicated than the POP3 protocol commands. We list a few examples below:

- `LOGIN <username> <password>` authenticates the user
- `SELECT <mailbox>` selects the mailbox folder to work with

- `FETCH <mail_number> <data_item_name>` Example `fetch 3 body[]` to fetch message number 3, header and body.
- `MOVE <sequence_set> <mailbox>` moves the specified messages to another mailbox
- `COPY <sequence_set> <data_item_name>` copies the specified messages to another mailbox
- `LOGOUT` logs out

Knowing that the IMAP server listens on TCP port 143 by default, we will use `telnet` to connect to `MACHINE_IP`'s port 143 and fetch the message we sent in an earlier task.

```
user@TryHackMe$ telnet 10.10.41.192 143
Trying 10.10.41.192...
Connected to 10.10.41.192.
Escape character is '^]'.
* OK [CAPABILITY IMAP4rev1 SASL-IR LOGIN-REFERRALS ID ENABLE I
DLE LITERAL+ STARTTLS AUTH=PLAIN] Dovecot (Ubuntu) ready.
A LOGIN strategos
A OK [CAPABILITY IMAP4rev1 SASL-IR LOGIN-REFERRALS ID ENABLE I
DLE SORT SORT=DISPLAY THREAD=REFERENCES THREAD=REFS THREAD=ORD
EREDSUBJECT MULTIAPPEND URL-PARTIAL CATENATE UNSELECT CHILDREN
NAMESPACE UIDPLUS LIST-EXTENDED I18NLEVEL=1 CONDSTORE QRESYNC
ESEARCH ESORT SEARCHRES WITHIN CONTEXT=SEARCH LIST-STATUS BINA
RY MOVE SNIPPET=FUZZY PREVIEW=FUZZY PREVIEW STATUS=SIZE SAVEDA
TE LITERAL+ NOTIFY SPECIAL-USE] Logged in
B SELECT inbox
* FLAGS (\Answered \Flagged \Deleted \Seen \Draft)
* OK [PERMANENTFLAGS (\Answered \Flagged \Deleted \Seen \Draft
\*)] Flags permitted.
* 4 EXISTS
* 0 RECENT
* OK [UNSEEN 2] First unseen.
* OK [UIDVALIDITY 1719824692] UIDs valid
* OK [UIDNEXT 5] Predicted next UID
B OK [READ-WRITE] Select completed (0.001 + 0.000 secs).
C FETCH 3 body[]
```

```

* 3 FETCH (BODY[] {445}
Return-path: <user@client.thm>
Envelope-to: strategos@server.thm
Delivery-date: Thu, 27 Jun 2024 16:19:35 +0000
Received: from [10.11.81.126] (helo=client.thm)
        by example.thm with smtp (Exim 4.95)
        (envelope-from <user@client.thm>)
        id 1sMrpq-0001Ah-UT
        for strategos@server.thm;
        Thu, 27 Jun 2024 16:19:35 +0000
From: user@client.thm
To: strategos@server.thm
Subject: Telnet email

Hello. I am using telnet to send you an email!
)
C OK Fetch completed (0.001 + 0.000 secs).
D LOGOUT
* BYE Logging out
D OK Logout completed (0.001 + 0.000 secs).
Connection closed by foreign host.

```

Task 8 - Conclusion

Protocol	Transport Protocol	Default Port Number
TELNET	TCP	23
DNS	UDP or TCP	53
HTTP	TCP	80
HTTPS	TCP	443
FTP	TCP	21
SMTP	TCP	25
POP3	TCP	110
IMAP	TCP	143

D. Networking Secure Protocol

Task 1 - Introduction

Transport Layer Security (TLS) is added to existing protocols to protect communication confidentiality, integrity, and authenticity.

Consequently, HTTP, POP3, SMTP, and IMAP become HTTPS, POP3S, SMTPS, and IMAPS, where the appended "S" stands for Secure. Similarly, it is deemed insecure to remotely access a system using the TELNET protocol; Secure Shell (SSH) was created to provide a secure way to access remote systems. Furthermore, SSH is an extensible protocol that offers added security features for other protocols.

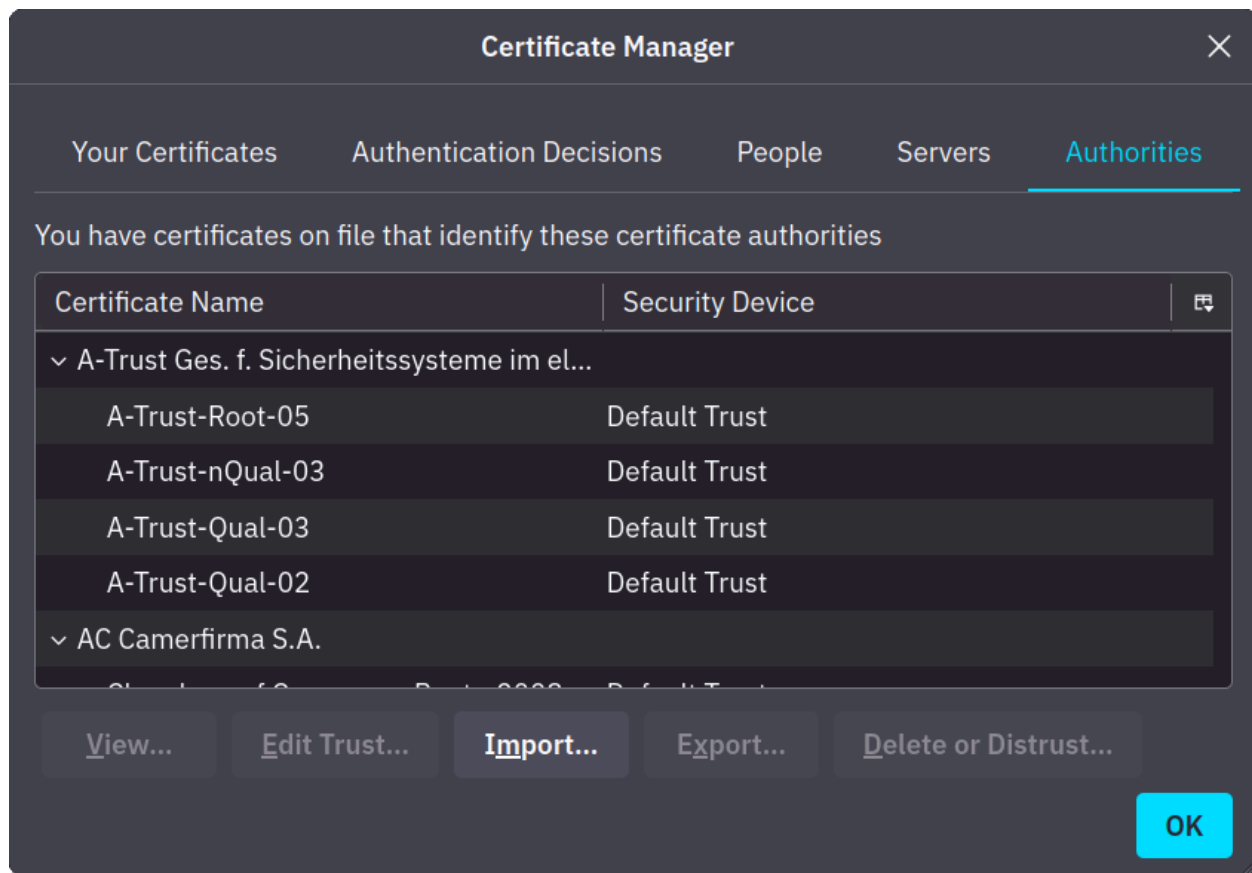
Task 2 - TLS

TLS is a security protocol that encrypts data in transit between two systems, ensuring confidentiality, integrity, and authentication. It is widely used in HTTPS to secure web traffic and prevent eavesdropping or tampering.

SSL/TLS is a cryptographic protocol used to secure network communication by encrypting data between a client and server. It provides confidentiality, integrity, and authentication, and is widely used in HTTPS to secure web traffic. TLS is the modern replacement for SSL.

Nowadays, tens of protocols have received security upgrades with the simple addition of TLS. Examples include HTTP, DNS, MQTT, and SIP, which have become HTTPS, DoT (DNS over TLS), MQTTS, and SIPS, where the appended "S" stands for Secure due to the use of SSL/TLS.

The first step for every server (or client) that needs to identify itself is to get a signed TLS certificate. Generally, the server administrator creates a Certificate Signing Request (CSR) and submits it to a Certificate Authority (CA); the CA verifies the CSR and issues a digital certificate. Once the (signed) certificate is received, it can be used to identify the server (or the client) to others, who can confirm the validity of the signature. For a host to confirm the validity of a signed certificate, the certificates of the signing authorities need to be installed on the host. In the non-digital world, this is similar to recognising the stamps of various authorities. The screenshot below shows the trusted authorities installed in a web browser.



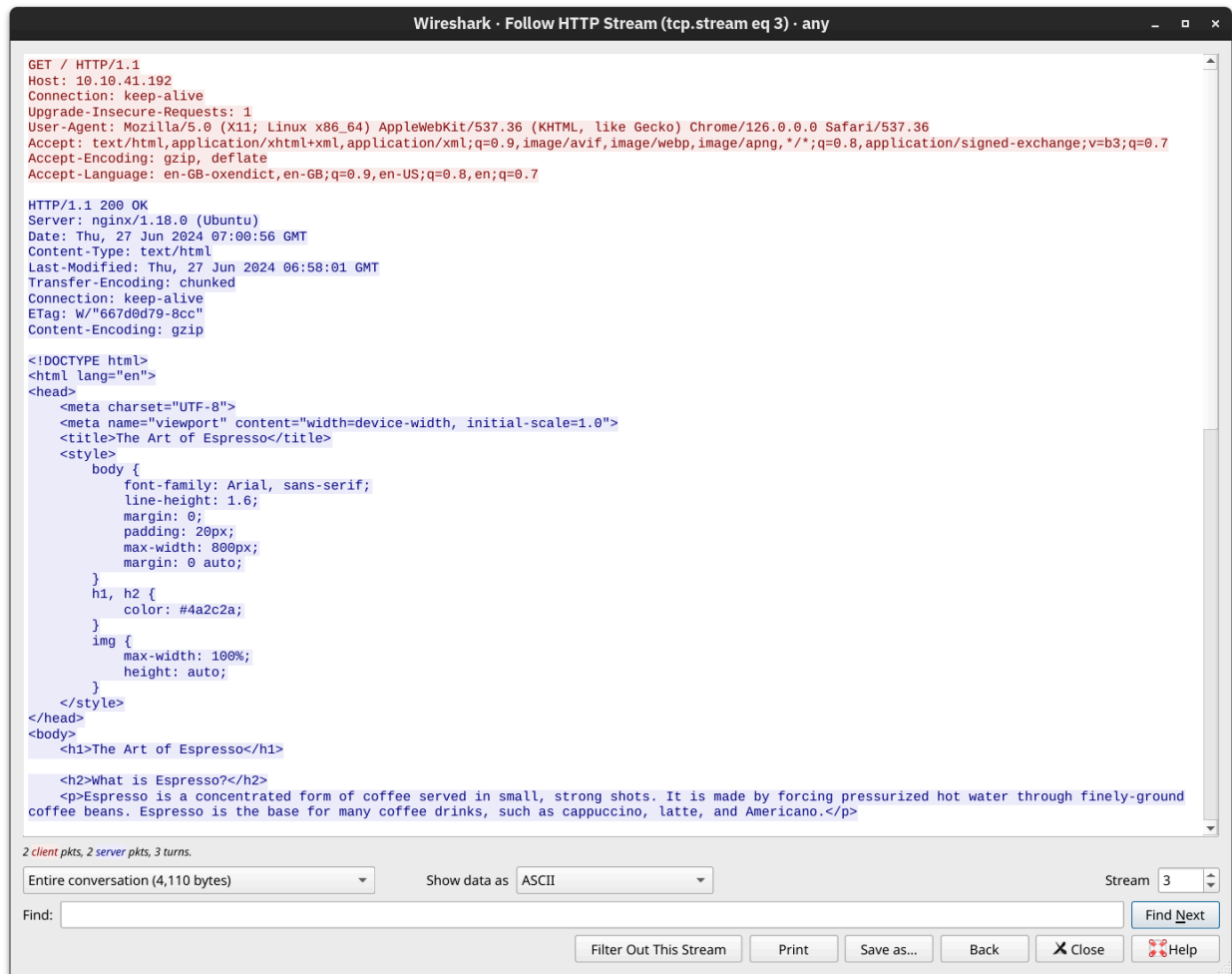
Answer the questions below

What is the protocol name that TLS upgraded and built upon? SSL

Which type of certificates should not be used to confirm the authenticity of a server? self-signed certificate

Task 3 - HTTPS

HTTP relies on TCP and uses port 80 by default. We also saw how all HTTP traffic was sent in cleartext for anyone to intercept and monitor. The screenshot below is from the previous room, and it gives a clear idea of how an adversary can easily read all the traffic exchanged between the client and the server.



```
GET / HTTP/1.1
Host: 10.10.41.192
Connection: keep-alive
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/126.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding: gzip, deflate
Accept-Language: en-GB;q=0.9,en-US;q=0.8,en;q=0.7

HTTP/1.1 200 OK
Server: nginx/1.18.0 (Ubuntu)
Date: Thu, 27 Jun 2024 07:00:56 GMT
Content-Type: text/html
Last-Modified: Thu, 27 Jun 2024 06:58:01 GMT
Transfer-Encoding: chunked
Connection: keep-alive
ETag: W/"667d8d79-8cc"
Content-Encoding: gzip

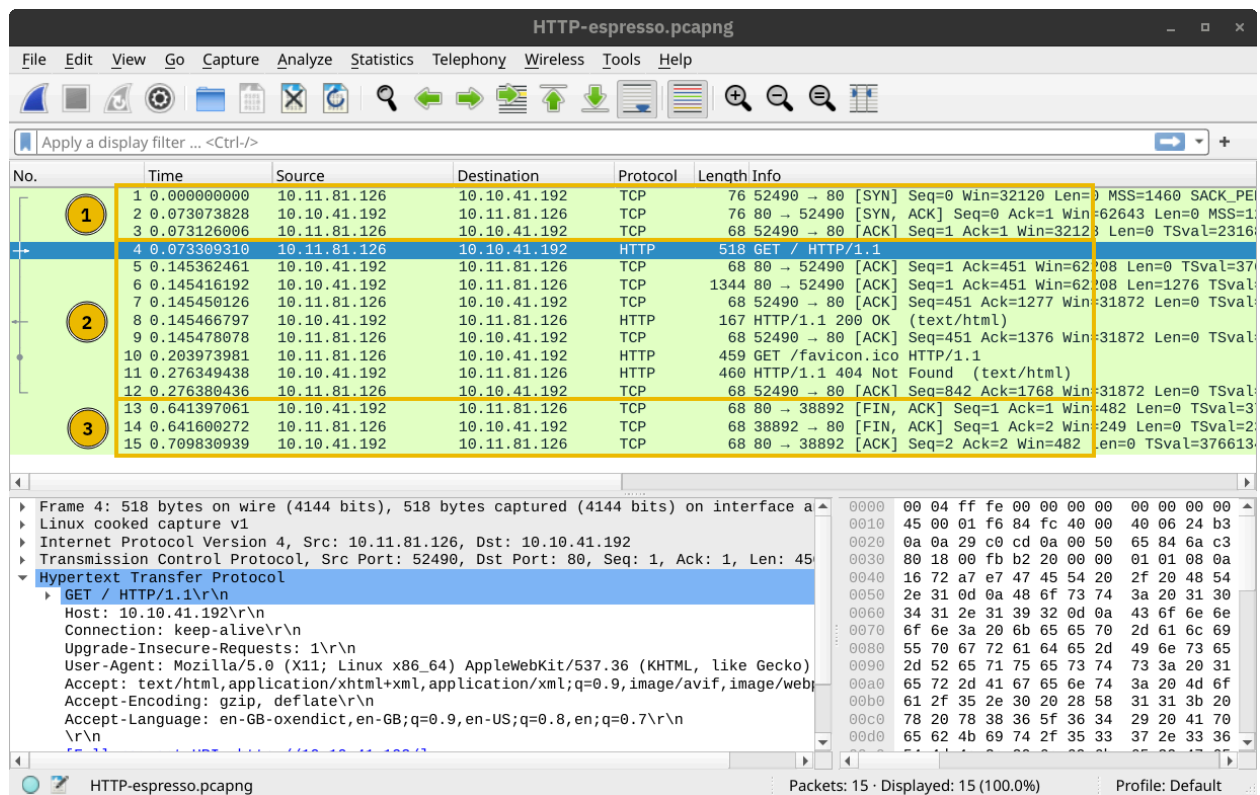
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>The Art of Espresso</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      line-height: 1.6;
      margin: 0;
      padding: 20px;
      max-width: 800px;
      margin: 0 auto;
    }
    h1, h2 {
      color: #4a2c2a;
    }
    img {
      max-width: 100%;
      height: auto;
    }
  </style>
</head>
<body>
  <h1>The Art of Espresso</h1>

  <h2>What is Espresso?</h2>
  <p>Espresso is a concentrated form of coffee served in small, strong shots. It is made by forcing pressurized hot water through finely-ground coffee beans. Espresso is the base for many coffee drinks, such as cappuccino, latte, and Americano.</p>
</body>
```

Let's take a minute to review the most common steps before a web browser can request a page over HTTP. After resolving the domain name to an IP address, the client will carry out the following two steps:

1. Establish a TCP three-way handshake with the target server
2. Communicate using the HTTP protocol; for example, issue HTTP requests, such as `GET / HTTP/1.1`

The two steps described above are shown in the window below. The three packets for the TCP handshake (marked with 1) precede the first HTTP packet with `GET` in it. The HTTP communication is marked with 2. The last three displayed packets are for TCP connection termination and are marked with 3.

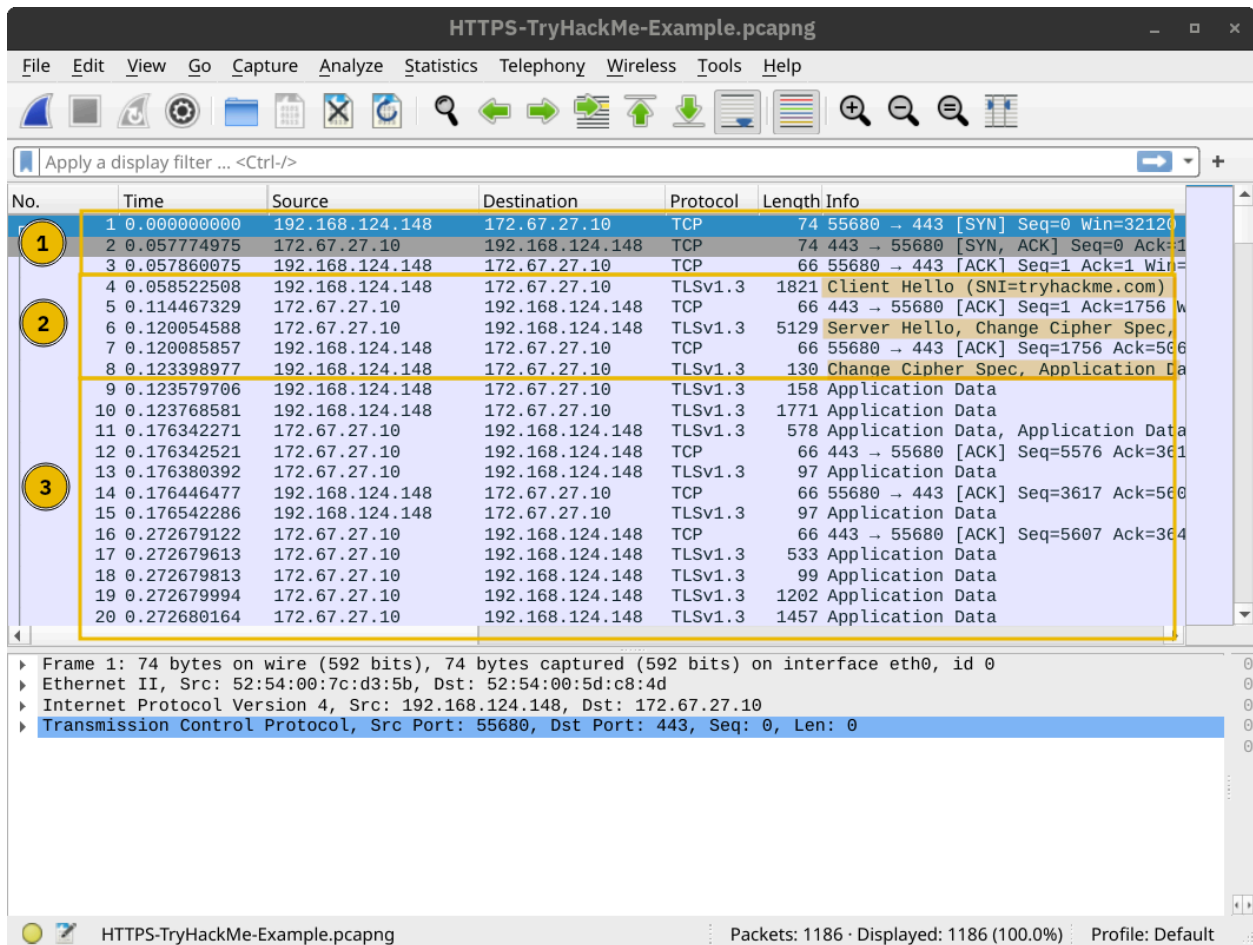


HTTP Over TLS: HTTPS stands for Hypertext Transfer Protocol Secure. It is basically HTTP over TLS. Consequently, requesting a page over HTTPS will require the following three steps (after resolving the domain name):

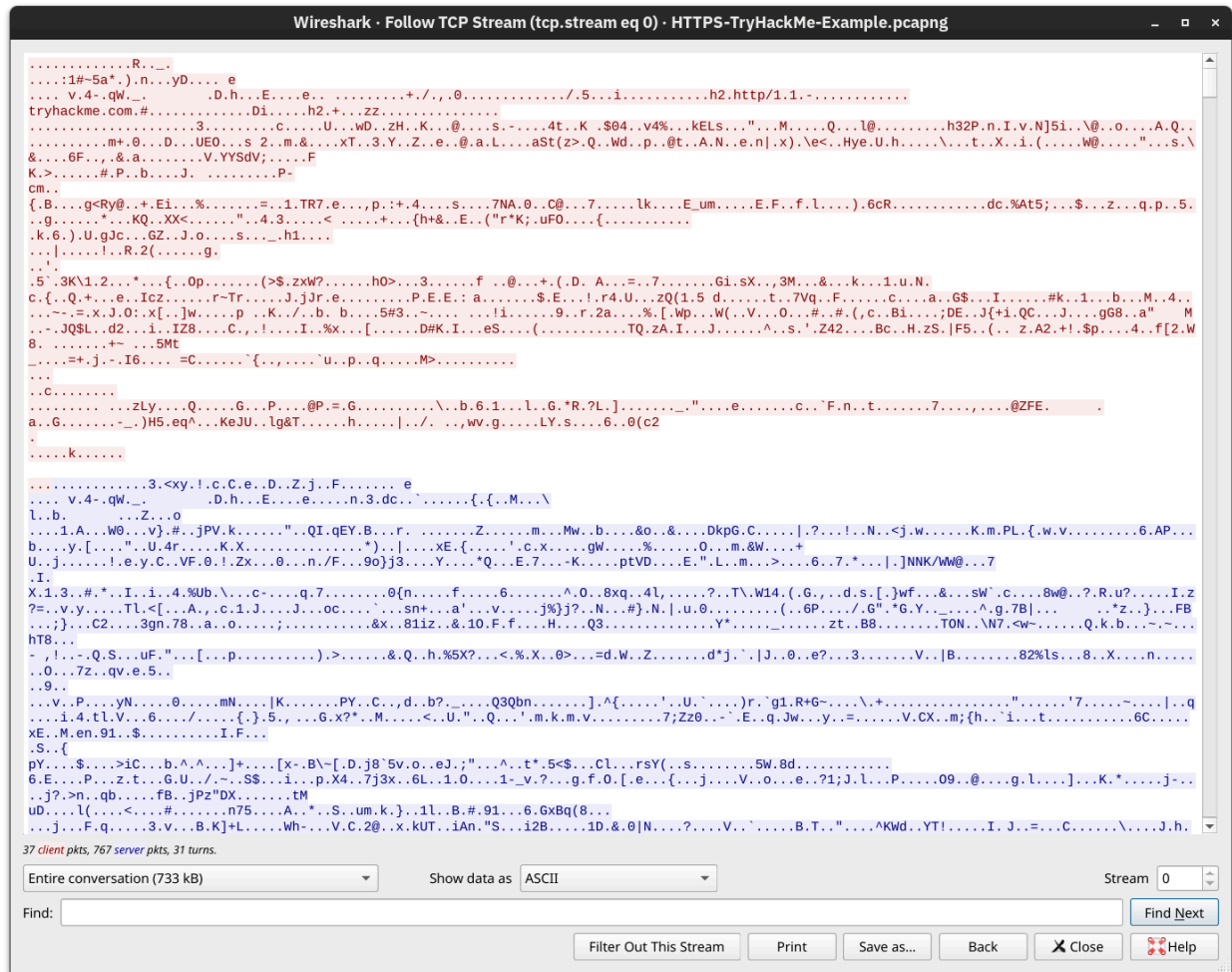
1. Establish a TCP three-way handshake with the target server
2. Establish a TLS session
3. Communicate using the HTTP protocol; for example, issue HTTP requests, such as `GET / HTTP/1.1`

The screenshot below shows that a TCP session is established in the first three packets, marked with **1**. Then, several packets are exchanged to negotiate the TLS protocol, marked with **2**. **1** and **2** are where the **TLS negotiation and establishment** take place.

Finally, HTTP application data is exchanged, marked with **3**. Looking at the Wireshark screenshot, we see that it says "Application Data" because there is no way to know if it is indeed HTTP or some other protocol sent over port 443.



As expected, if one tries to follow the stream of packets and combine all their contents, they will only get gibberish, as shown in the screenshot below. The exchanged traffic is encrypted; the red is sent by the client, and the blue is sent by the server. There is no way to know the contents without acquiring the encryption key.



Getting the Encryption Key: Adding TLS to HTTP leads to all the packets being encrypted. We can no longer see the contents of the exchanged packets unless we get access to the private key. Although it is improbable that we will have access to the keys used for encryption in a TLS session, we repeated the above screenshots after providing the decryption key to Wireshark. The TCP and TLS handshakes don't change; the main difference starts with the HTTP protocol marked 3. For instance, we can see when the client issues a **GET**.

HTTPS pcapng

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

ip.addr == 172.67.27.10

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	192.168.124.148	172.67.27.10	TCP	74	55680 → 443 [SYN] Seq=0 Win=32123
2	0.057774975	172.67.27.10	192.168.124.148	TCP	74	443 → 55680 [SYN, ACK] Seq=0 Ack=1
3	0.057860075	192.168.124.148	172.67.27.10	TCP	66	55680 → 443 [ACK] Seq=1 Ack=1 Win=
4	0.058522508	192.168.124.148	172.67.27.10	TLSv1.3	1821	Client Hello (SNI=tryhackme.com)
5	0.114467329	172.67.27.10	192.168.124.148	TCP	66	443 → 55680 [ACK] Seq=1 Ack=1756 w
6	0.120054588	172.67.27.10	192.168.124.148	TLSv1.3	5129	Server Hello, Change Cipher Spec.
7	0.120085857	192.168.124.148	172.67.27.10	TCP	66	55680 → 443 [ACK] Seq=1756 Ack=536
8	0.123398977	192.168.124.148	172.67.27.10	TLSv1.3	130	Change Cipher Spec, Finished
9	0.123579706	192.168.124.148	172.67.27.10	HTTP2	158	Magic, SETTINGS[0], WINDOW_UPDATE[0]
10	0.123768581	192.168.124.148	172.67.27.10	HTTP2	1771	HEADERS[1]: GET /login
11	0.176342271	172.67.27.10	192.168.124.148	HTTP2	578	SETTINGS[0], WINDOW_UPDATE[0]
12	0.176342521	172.67.27.10	192.168.124.148	TCP	66	443 → 55680 [ACK] Seq=5576 Ack=331
13	0.176380392	172.67.27.10	192.168.124.148	HTTP2	97	SETTINGS[0]
14	0.176446477	192.168.124.148	172.67.27.10	TCP	66	55680 → 443 [ACK] Seq=3617 Ack=536
15	0.176542286	192.168.124.148	172.67.27.10	HTTP2	97	SETTINGS[0]
16	0.272679122	172.67.27.10	192.168.124.148	TCP	66	443 → 55680 [ACK] Seq=5607 Ack=334
17	0.272679613	172.67.27.10	192.168.124.148	HTTP2	533	HEADERS[1]: 200 OK
18	0.272679813	172.67.27.10	192.168.124.148	HTTP2	99	DATA[1]
19	0.272679994	172.67.27.10	192.168.124.148	HTTP2	1202	DATA[1]
20	0.272680164	172.67.27.10	192.168.124.148	TLSv1.3	1457	[TLS segment of a reassembled PDJ]

Frame 10: 1771 bytes on wire (14168 bits), 1771 bytes captured (14168 bits) on interface eth0, id 0

Ethernet II, Src: 52:54:00:7c:d3:5b, Dst: 52:54:00:5d:c8:4d

Internet Protocol Version 4, Src: 192.168.124.148, Dst: 172.67.27.10

Transmission Control Protocol, Src Port: 55680, Dst Port: 443, Seq: 1912, Ack: 5064, Len: 1705

Transport Layer Security

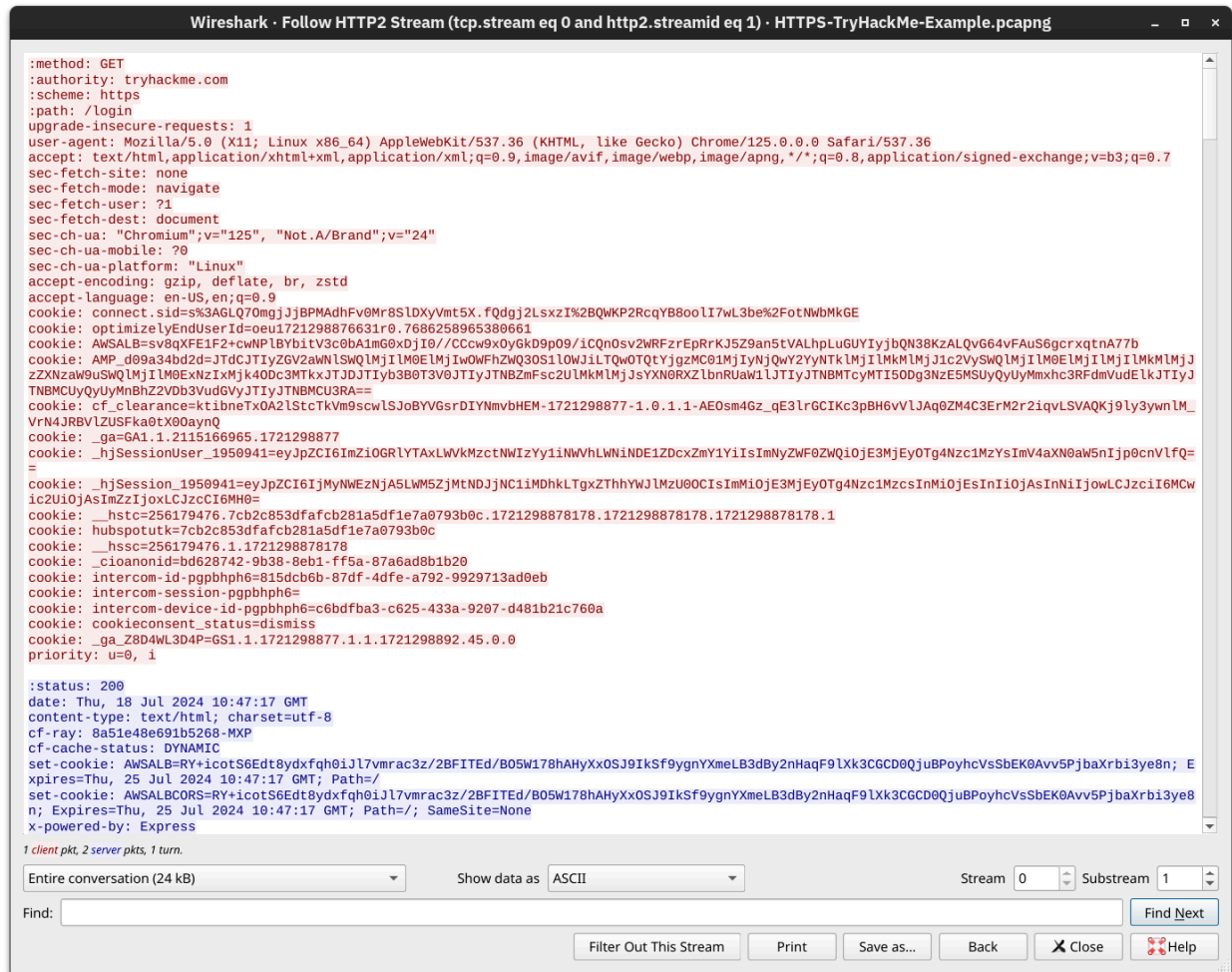
HyperText Transfer Protocol 2

- Stream: HEADERS, Stream ID: 1, Length 1674, GET /login
 - Length: 1674
 - Type: HEADERS (1)
 - Flags: 0x25, Priority, End Headers, End Stream
 - 0... .. = Reserved: 0x0
 - .000 0000 0000 0000 0000 0000 0000 0001 = Stream Identifier: 1
 - Pad Length: 01

HTTPS.pcapng

Packets: 1186 · Displayed: 1186 (100.0%) Profile: Default

If you want to see the data exchanged, now is your chance! It is still regular HTTP traffic hidden from prying eyes.



The key takeaway is that TLS offered security for HTTP without requiring any changes in the lower or higher layer protocols. In other words, TCP and IP were not modified, while HTTP was sent over TLS the way it would be sent over TCP.

Answer the questions below

How many packets did the TLS negotiation and establishment take in the Wireshark HTTPS screenshots above? 8

What is the number of the packet that contain the GET /login when accessing the website over HTTPS? 10

Task 4 - SMTPS, POP3S, and IMAPS

Adding TLS to SMTP, POP3, and IMAP is no different than adding TLS to HTTP. Similar to how HTTP gets an appended S for *Secure* and becomes

HTTPS, SMTP, POP3, and IMAP become SMTPS, POP3S, and IMAPS, respectively. Using these protocols over TLS is no different than using HTTP over TLS; therefore, almost all the points from the HTTPS discussion apply to these protocols. The insecure versions use the default TCP port numbers shown in the table below:

Protocol	Default Port Number
HTTP	80
SMTP	25
POP3	110
IMAP	143

The secure versions, i.e., over TLS, use the following TCP port numbers by default:

Protocol	Default Port Number
HTTPS	443
SMTPS	465 and 587
POP3S	995
IMAPS	993

TLS can be added to many other protocols; the reasoning and advantages would be similar.

If you capture network traffic, in which of the following protocols can you extract login credentials: SMTPS, POP3S, or IMAP? IMAP

Task 5 - SSH

Although telnet is very convenient to log in and administer remote systems, it is risky when all the traffic is sent in cleartext. It is easy for anyone monitoring the network traffic to get hold of your login credentials once you use `telnet`. This problem necessitated a solution. OpenSSH offers several benefits. We will list a few key points:

- **Secure authentication:** Besides password-based authentication, SSH supports public key and two-factor authentication.
- **Confidentiality:** OpenSSH provides end-to-end encryption, protecting against eavesdropping. Furthermore, it notifies you of new server keys to protect against man-in-the-middle attacks.

- **Integrity:** In addition to protecting the confidentiality of the exchanged data, cryptography also protects the integrity of the traffic.
- **Tunneling:** SSH can create a secure “tunnel” to route other protocols through SSH. This setup leads to a VPN-like connection.
- **X11 Forwarding:** If you connect to a Unix-like system with a graphical user interface, SSH allows you to use the graphical application over the network.

You would issue the command `ssh username@hostname` to connect to an SSH server. If the username is the same as your logged-in username, you only need `ssh hostname`. Then, you will be asked for a password; however, if public-key authentication is used, you will be logged in immediately. While the TELNET server listens on port 23, the SSH server listens on port 22.

What is the name of the open-source implementation of the SSH protocol?
OpenSSH

Task 6 - SFTP and FTPS

SFTP stands for SSH File Transfer Protocol and allows secure file transfer. It is part of the SSH protocol suite and shares the same port number, 22. If enabled in the OpenSSH server configuration, you can connect using a command such as `sftp username@hostname`. Once logged in, you can issue commands such as `get filename` and `put filename` to download and upload files, respectively. Generally speaking, SFTP commands are Unix-like and can differ from FTP commands.

SFTP should not be confused with FTPS. You are right to think that FTPS stands for File Transfer Protocol Secure. How is FTPS secured? Yes, you are correct to estimate that it is secured using TLS, just like HTTPS. While FTP uses port 21, FTPS usually uses port 990. It requires certificate setup, and it can be tricky to allow over strict firewalls as it uses separate connections for control and data transfer.

Setting up an SFTP server is as easy as enabling an option within the OpenSSH server. Like HTTPS, SMTPS, POP3S, IMAPS, and other protocols that rely on TLS for security, FTPS requires a proper TLS certificate to run securely.

Task 7 - VPN

A VPN (Virtual Private Network) creates an encrypted tunnel between a user's device and a remote network. It is used to secure data in transit, protect users on untrusted networks, and provide remote access to corporate resources. Common VPN technologies include IPsec, OpenVPN, WireGuard, and SSL/TLS VPNs.

Types of VPN

1. Remote Access VPN: Allows individual users to connect to a corporate network remotely.

Example: Employee Home PC—VPN Tunnel —Corporate Network

Commonly used for: Work from home, Remote administration, Secure access to internal applications

2. Site-to-Site VPN: Connects entire networks together.

Example: Branch Office — VPN — Headquarters

Used by organizations with multiple locations.

Common VPN Protocols

Protocol	Description
IPsec	Most common enterprise VPN protocol
SSL/TLS VPN	Uses TLS encryption (browser-based or client-based)
OpenVPN	Open-source VPN solution
WireGuard	Modern, lightweight, high-performance VPN
L2TP/IPsec	Older VPN solution

Feature	VPN	HTTPS
Encrypts traffic	All traffic to VPN server	Only web traffic to a specific site
Scope	Entire device/network connection	Individual website connection
Common Protocols	IPsec, WireGuard, OpenVPN	TLS
Main Use	Secure remote access	Secure web browsing

What would you use to connect the various company sites so that users at a remote office can access resources located within the main branch? VPN

Q: What is a VPN?

A VPN is a technology that creates an encrypted tunnel between a user and a network, allowing secure communication over the internet.

Q: Why do organizations use VPNs?

To provide secure remote access to internal resources and protect data in transit.

Q: What protocol is commonly used for VPNs?

IPsec, SSL/TLS VPN, OpenVPN, and WireGuard are common VPN technologies.

Q: Does a VPN encrypt traffic?

Yes, the traffic between the VPN client and VPN server is encrypted.

Q: What is the difference between VPN and HTTPS?

HTTPS encrypts communication between a browser and a website, while a VPN encrypts all network traffic between a device and the VPN server.